# EE200: Signals, Systems and Networks
## Course Project - Q1A: Frequency Forensics
## "The Ghost Signal"

# 1 Problem Statement

This question gives us a grayscale image that supposedly has some important information hidden in it. The catch is that the image has been corrupted on purpose - there is a repeating dot-grid pattern stamped over the whole thing, so just by looking at it you cannot make out anything. If you open the raw image you basically just see a dense grid of dots everywhere. At first glance it looks like the image is ruined for good.

But the trick here is that periodic interference and actual image content behave very differently once we move to the frequency domain. A repeating pattern, no matter how strong it looks visually, ends up concentrated at a few specific frequencies, and that means it can be removed without disturbing the rest of the image.

So our plan for this question is:

- Take the corrupted image and compute its 2D DFT.
- Look at the spectrum and figure out where the noise is sitting.
- Build a notch filter that knocks out only those noise frequencies.
- Take the inverse DFT and get the cleaned-up image back.
- Check what happens if we change how aggressive the filter is.

The image we were given, ghost_signal_input.png, is a grayscale picture that on the surface just looks like a dense regular grid of dots - whatever text is underneath is completely invisible to the eye.

# 2 Background: The 2D Discrete Fourier Transform

## 2.1 Why bother with the Fourier Transform at all?

In the spatial domain (i.e. the actual pixel grid) the dot pattern and the text are just added together pixel by pixel, so there is no way to tell which part of a pixel's value comes from the text and which from the noise. They're stuck together.

The Fourier Transform gives us a different way to look at the same picture. Instead of asking what the brightness is at pixel $(x, y)$, it asks how much of each spatial frequency is present in the image overall. This matters because the dot grid is periodic, so almost all of its energy piles up at just a few frequencies. The text, on the other hand, is not periodic at all, so its energy is smeared out over a wide range of frequencies. In other words, even though they overlap in the spatial domain, they end up sitting in different places once we move to the frequency domain, and that's really the whole reason this method works.

## 2.2 The 2D DFT formula

For an $M \times N$ grayscale image $I(x, y)$, with $x$ running over rows (0 to $M-1$) and $y$ over columns (0 to $N-1$), the 2D DFT is:

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} I(x,y) \cdot \exp\left(-j \cdot 2\pi \cdot \left(\frac{ux}{M} + \frac{vy}{N}\right)\right)$$

Quick rundown of what each symbol means:

- $I(x, y)$ - the brightness value of the pixel at row $x$, column $y$ (0 = black, 255 = white).
- $(u, v)$ - the frequency indices. $u$ tells you how many oscillations happen across the width, $v$ across the height.

- $\exp\left(-j \cdot 2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)\right)$ - a complex sinusoid oscillating at $(u/M, v/N)$. By Euler's formula it's $\cos\left(2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)\right) - j \cdot \sin\left(2\pi \left(\frac{ux}{M} + \frac{vy}{N}\right)\right)$.
- $F(u, v)$ - a complex number for every $(u, v)$. Its magnitude says how strong that frequency is, its phase tells you the spatial shift of that component.

### 2.3 The inverse 2D DFT

Once we've edited the spectrum $F(u, v)$ to remove the noise, we need a way to come back to the image domain. That's what the inverse DFT does:

$$I(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \exp\left(+j \cdot 2\pi \cdot \left(\frac{ux}{M} + \frac{vy}{N}\right)\right)$$

### 2.4 Is the spectrum naturally centered?

No. When we call `np.fft.fft2(img)`, numpy places the zero-frequency (DC) term at the top-left corner of the array, $F[0, 0]$. Frequency increases as you move right/down and then wraps around, so the spectrum visually looks split into four chunks that don't line up the way you'd expect.

To fix this we use `np.fft.fftshift()`, which swaps the four quadrants around so DC lands exactly in the middle. After that, low frequencies are at the centre and higher frequencies spread out symmetrically - it ends up looking like a bullseye, which is much easier to interpret. Before doing the inverse transform we obviously have to undo this with `np.fft.ifftshift()` so the DC term is back where `ifft2()` expects it.

## 3 Step 1 - Loading the Image and Computing the Spectrum

### 3.1 Implementation

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Load image and convert to grayscale
img_rgb = Image.open('ghost_signal_input.png').convert('L')
img = np.array(img_rgb, dtype=np.float64)
H, W = img.shape

# Compute 2D DFT and shift DC to center
F = np.fft.fft2(img)
F_shifted = np.fft.fftshift(F)

# Magnitude in linear and dB scale
magnitude_linear = np.abs(F_shifted)
magnitude_db = 20 * np.log10(magnitude_linear + 1e-6)
```

A couple of choices here are worth explaining. `.convert('L')` turns whatever colour image we have into a single grayscale channel using $L = 0.299R + 0.587G + 0.114B$ (this weighting matches how sensitive our eyes are to each colour). We also cast everything to `float64` right away, because the DFT involves complex and fractional values that would just get chopped off if we left the array as integers.

The magnitude is $|F(u, v)| = \sqrt{a^2 + b^2}$, where $a$ and $b$ are the real and imaginary parts - `np.abs()` does this for us directly on the complex array. For the dB version we use $20 \cdot \log_{10}(|F|)$, with a tiny `1e-6` added before the log so we never take $\log(0)$.

### 3.2 What we see in the spectrum

On the linear scale, almost the entire spectrum looks black except for a few extremely bright points that form a cross-like pattern through the centre. That happens because the dot grid carries way more energy than any single frequency belonging to the actual image. So on a linear scale, the noise spikes basically

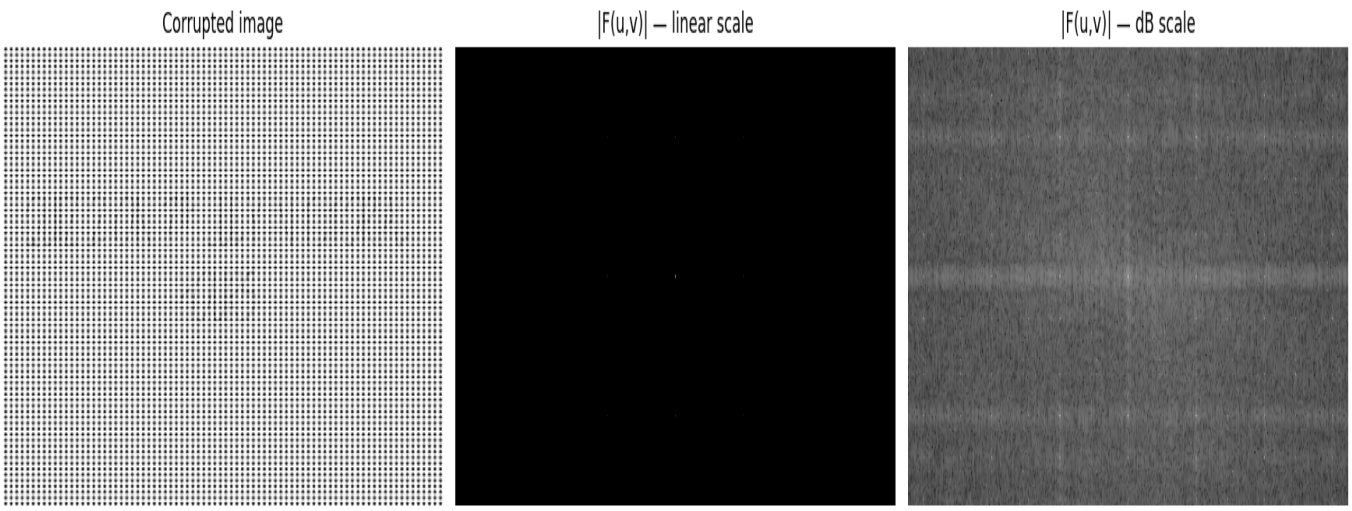


Figure 1: IMAGE

drown out everything else, kind of like trying to spot stars during the day - they're there, but the sun is just too bright.

Switching to dB changes things a lot, since the log compresses the strong and weak parts into a range we can actually look at. Now we can see two separate things: a broad, smooth cloud of energy sitting around the DC point - this is the actual image content (smooth regions near DC, edges and text boundaries spreading further out) - and a handful of small, very bright, isolated dots placed symmetrically on top of that cloud. Those isolated dots are the interference. This contrast is basically the whole diagnostic: real content looks smooth and spread out, periodic noise looks like sharp pinpoints.

# 4 Step 2 - Locating the Interference Spikes

## 4.1 Why does periodic noise show up as point-like spikes?

Before getting into the code, it helps to understand why a periodic dot grid produces these isolated spikes, since this is really the core idea behind the whole question.

A pattern that repeats every $P$ pixels horizontally can be written as a sum of sinusoids at a fundamental frequency $f_0 = 1/P$ plus harmonics at $2f_0$, $3f_0$ and so on. Each of these sinusoids only puts energy at one specific bin in the DFT and nowhere else. So a grid that repeats both horizontally and vertically only puts energy at the fundamental and its multiples in both directions, which gives a sparse, symmetric grid of spikes in the 2D spectrum.

Actual image content (text, shading, edges) doesn't repeat at any fixed period across the whole image, so it can't concentrate its energy into a few bins - it has to spread out continuously. That's basically why noise and signal look so different once you transform to frequency.

Also, since our image is real-valued, the spectrum has conjugate symmetry: $F(-u, -v) = F^*(u, v)$. So every noise spike at $(u_0, v_0)$ has a mirror spike of equal magnitude at $(-u_0, -v_0)$. We'll always see the spikes in symmetric pairs around the centre.

## 4.2 Peak detection code

```
cy, cx = H//2, W//2
Y, X = np.mgrid[0:H, 0:W]
dist_from_center = np.sqrt((X-cx)**2 + (Y-cy)**2)

# work on a copy, never touch the original
search_region = magnitude_db.copy()

# mask out genuine low-frequency content near DC
search_region[dist_from_center < 5] = -np.inf
```

```
# find brightest well-separated peaks
flat_idx = np.argsort(search_region.ravel())[::-1]
peak_coords = []

for idx in flat_idx:
    y, x = np.unravel_index(idx, search_region.shape)
    if search_region[y, x] == -np.inf:
        continue

    too_close = any(abs(y-py) < 5 and abs(x-px) < 5 for py, px in peak_coords)
    if not too_close:
        peak_coords.append((y, x))

    if len(peak_coords) >= 20:
        break
```

We get `(cy, cx)` for the DC location in the shifted spectrum, then compute the distance of every pixel from that point. Instead of looping over pixels in plain Python (slow), we use `np.mgrid` to build coordinate grids `X` and `Y` so the whole distance computation happens in one vectorised line.

We block out a small 5-pixel disk around the centre before searching, because the DC term and the genuine low-frequency content (smooth areas of the image) are also bright in dB. If we didn't mask this region, the peak finder would mistake them for noise and try to remove them, which would mess up the overall brightness and large-scale structure of the image.

The search itself sorts pixels by brightness (descending) and walks through them, keeping a peak only if it is at least 5 pixels away from every peak already found - this stops us from picking up several pixels off the same spike as if they were separate peaks (basically non-maximum suppression). We cap it at 20 peaks, which is plenty to capture the fundamental plus the first couple of harmonics without accidentally flagging real image frequencies.

### 4.3 Detected peaks

| Array Row | Array Col | Freq u | Freq v | Magnitude (dB) |
|---|---|---|---|---|
| 52 | 258 | 0 | -80 | 133.9 |
| 212 | 258 | 0 | +80 | 133.9 |
| 132 | 338 | +80 | 0 | 133.9 |
| 132 | 178 | -80 | 0 | 133.9 |
| 212 | 178 | -80 | +80 | 128.3 |
| 132 | 418 | +160 | 0 | 117.9 |
| 236 | 258 | 0 | +104 | 117.8 |

Table 1: Table 1: Detected interference peaks (shortened - full list of 20 peaks is in the notebook). Frequencies are given relative to DC at the centre.

The peaks line up in a clean, symmetric pattern. The strongest ones (133.9 dB) sit at $(u, v) = (\pm 80, 0)$ and $(0, \pm 80)$ - these are the fundamental frequencies of the dot grid horizontally and vertically. That corresponds to the grid repeating roughly every $H/80$ pixels vertically and $W/80$ pixels horizontally, so a spatial period of around 4-8 pixels depending on image size.

There's a weaker set of peaks around $(u, v) = (\pm 160, 0)$ and $(0, \pm 104)$ at about 117.9 dB - these are second harmonics. They show up because the dot pattern isn't a clean sine wave (real dots have hard edges), and any non-sinusoidal periodic signal splits into a fundamental plus harmonics. Since each harmonic is weaker than the last, we only really need to remove the first few.

Every spike also has a mirror partner (e.g. $(0, +80)$ and $(0, -80)$) which checks out with the conjugate symmetry we'd expect for a real image.

# 5 Step 3 - Notch Filter Design

## 5.1 What is a notch filter, and why use one here?

A notch filter is just a mask in the frequency domain that passes every frequency except a few specific spots - the "notches" - where it sets the gain to zero. Everywhere it's 1 (pass through), and at the noise locations it's 0 (blocked).

It's the right tool here because the noise only occupies a small number of isolated bins. A low-pass filter would wipe out everything above some cutoff frequency, but our spikes aren't confined to high frequencies only - they show up at specific spots across the whole range. A band-stop filter would remove a whole ring of frequencies, which is way too blunt. The notch filter is more surgical: it kills exactly the bins we've identified and leaves everything else alone.

## 5.2 Implementation

```
base_u, base_v = 80, 80 # fundamental from Step 2
peak_freqs_uv = set()

# build set of all harmonics to notch
for k in range(0, 4):
    for l in range(0, 4):
        peak_freqs_uv.add((k*base_u, l*base_v))
        peak_freqs_uv.add((k*base_u, -l*base_v))

peak_freqs_uv.discard((0, 0)) # never touch DC
peak_freqs_uv = list(peak_freqs_uv)

def make_notch_filter(radius, freqs):
    notch = np.ones((H, W), dtype=np.float64) # start: pass everything
    for (u, v) in freqs:
        for sign in [1, -1]:
            uc = cx + sign*u
            vc = cy + sign*v
            if 0 <= uc < W and 0 <= vc < H:
                # conjugate mirror
                mask = (X-uc)**2 + (Y-vc)**2 <= radius**2
                notch[mask] = 0.0
    return notch

r = 5
notch = make_notch_filter(r, peak_freqs_uv)
F_filtered_shifted = F_shifted * notch
```

We start the filter as all ones (everything passes), then punch circular holes of zeros at the noise locations. We use disks instead of single points because each spike isn't perfectly concentrated at one pixel - it has a small spread around it, just from the finite size of the image. A radius-5 disk is enough to fully cover the spike and the area immediately around it.

The `k, l` loop generates harmonics up to the 3rd order, in the horizontal, vertical, and diagonal directions. This is needed because a 2D periodic pattern produces energy at all combinations of its horizontal and vertical harmonics - not just the pure horizontal/vertical ones. If we only notched the axis-aligned spikes, the diagonal harmonics would survive and leave a visible residue in the recovered image.

We explicitly discard $(0, 0)$ so we never zero the DC term, since that's just the average brightness of the whole image - zeroing it would force the recovered image's mean to zero and wreck the contrast. For every frequency we also notch its conjugate mirror (the `for sign in [1, -1]` loop); if we only handled one of the pair, the inverse DFT result wouldn't stay real-valued, and we'd get residual junk back in the recovered image.

### 5.3 Why these frequencies?

Notching at $(u = k \times 80, v = l \times 80)$ for $k, l = 0..3$ isn't a random guess - it comes straight from the peak detection in Step 2, which gave us a fundamental of $\pm 80$ in both directions. Going up to the 3rd harmonic captures the first three repeats of the pattern's energy.

Past the 3rd harmonic the energy drops off quickly, so removing the 4th harmonic wouldn't visibly change the result. At the same time, going further out starts to risk overlapping with real image content (fine text edges live in that region too). So stopping at $k = 3$ is a deliberate compromise - enough harmonics to kill the visible grid without touching the fine detail of the image.

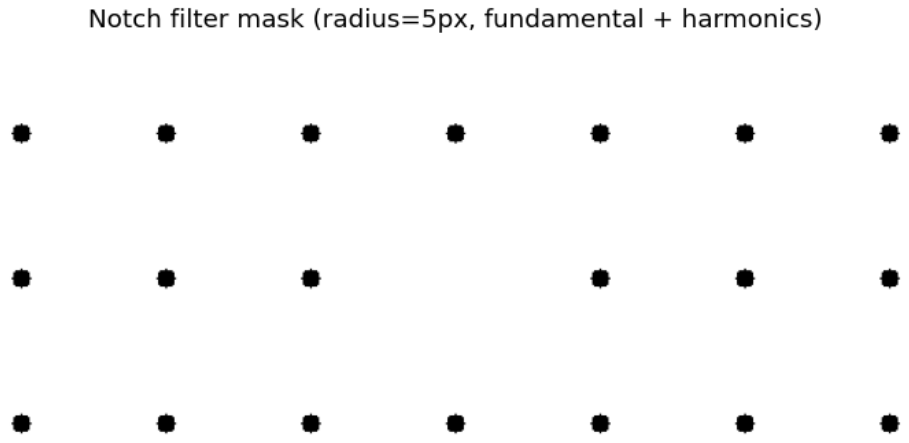


Figure 2: IMAGE

## 6 STEP 4. IDTFT IMAGE RECOVERY

### 6.1 Implementation

```
# Undo the fftshift before applying inverse DFT
F_filtered = np.fft.ifftshift(F_filtered_shifted)

# Apply inverse 2D DFT
recovered = np.fft.ifft2(F_filtered)

# Discard negligible floating-point imaginary residual
recovered = np.real(recovered)

# Contrast stretch for display only
lo, hi = np.percentile(recovered, 1), np.percentile(recovered, 99)
recovered_display = np.clip((recovered - lo) / (hi - lo), 0, 1)
```

Calling `ifftshift` before `ifft2` is a step that's very easy to forget, but it's essential. All the work up to now (peak detection, filter, multiplication) was done on the centre-shifted spectrum, but `np.fft.ifft2` expects DC to be at the top-left corner, the standard layout. If we fed it the shifted spectrum directly the output would be scrambled garbage. `ifftshift` moves the quadrants back before we inverse-transform.

In theory the output of `ifft2` should be purely real, since we started from a real image and the notch mask is symmetric, which preserves conjugate symmetry. In practice floating-point rounding errors build up over the millions of operations in the DFT, leaving an imaginary part around $10^{-10}$ or smaller - essentially noise, so we just keep `np.real()`.

The contrast stretch at the end is purely for display - we map the 1st-99th percentile range to $[0, 1]$ so the displayed image looks normal. We use percentiles rather than the absolute min/max so a few

stray outlier pixels near the notch edges don't wreck the scaling. The actual recovered array itself isn't touched by this.

### 6.2 Results - full pipeline

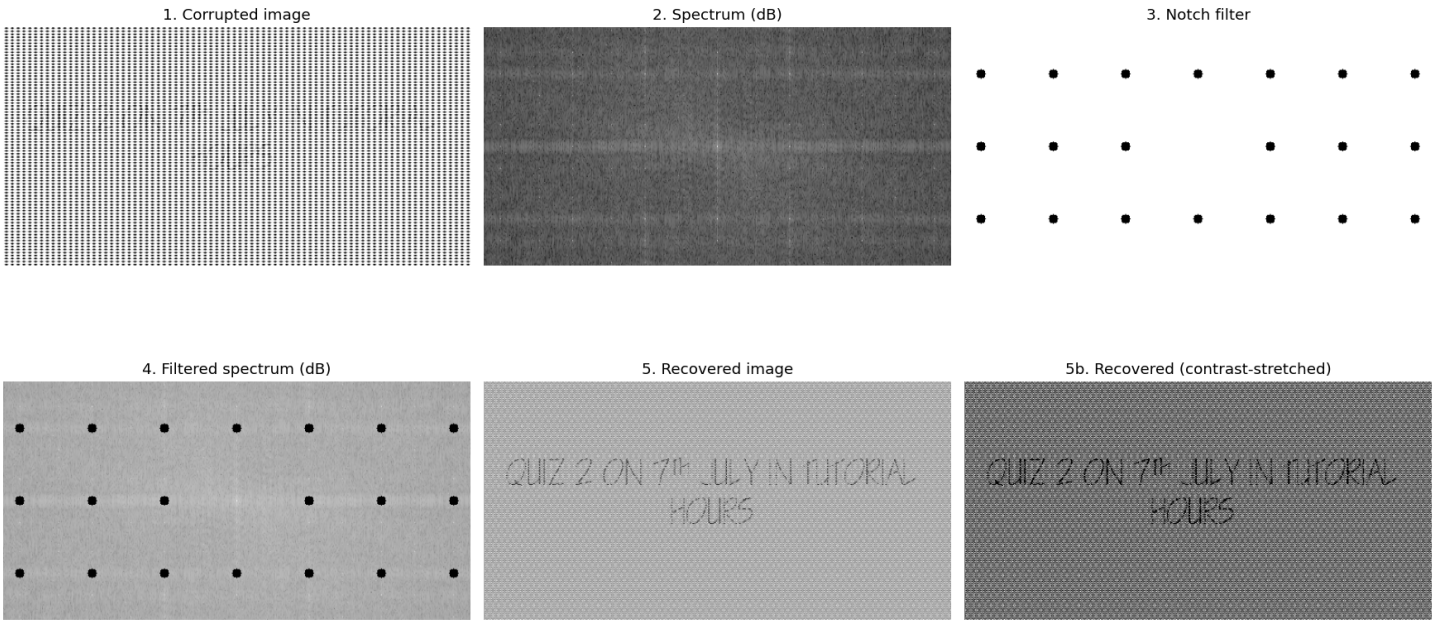


Figure 3: [Figure 3 goes here - $2 \times 3$ grid: (1) corrupted image, (2) dB spectrum, (3) notch filter, (4) filtered spectrum, (5)/(5b) recovered image before/after contrast stretching]

Comparing the corrupted input with the recovered output, the difference is dramatic. The dense dot grid that completely hid the image is basically gone, and the message underneath comes through clearly: "QUIZ 2 ON 7th JULY IN TUTORIAL HOURS"

Looking at the filtered spectrum, the bright isolated dots that were visible before are now gone (replaced by black holes), while the broad cloud of energy in the middle (the actual image content) is untouched. That confirms the filter only removed the noise and nothing else.

## 7 Step 5 - Effect of Filter Radius on Recovery Quality

### 7.1 The core trade-off

Picking $r = 5$ for the notch radius wasn't random - there's a real tension to balance here. The hole has to be big enough to fully cover a spike and its immediate surroundings, but small enough that it doesn't start eating into the real image frequencies nearby. There isn't really a clean closed-form answer for this; it needs some empirical tuning guided by understanding what's actually going on.

To see this properly we tried four radii and compared the recovered images side by side.

```
radii_to_try = [1, 5, 15, 35]
for rad in radii_to_try:
    nf = make_notch_filter(rad, peak_freqs_uv)
    rec = np.real(np.fft.ifft2(np.fft.ifftshift(F_shifted * nf)))
    # contrast stretch and display
```

### 7.2 What we observed

- **radius = 1px - not enough coverage**: With a 1-pixel radius the hole is smaller than the actual spread of each spike's energy. The very centre gets zeroed but a thin ring of residual noise survives around the edge of the hole. When we inverse-transform, those leftover rings recombine into a

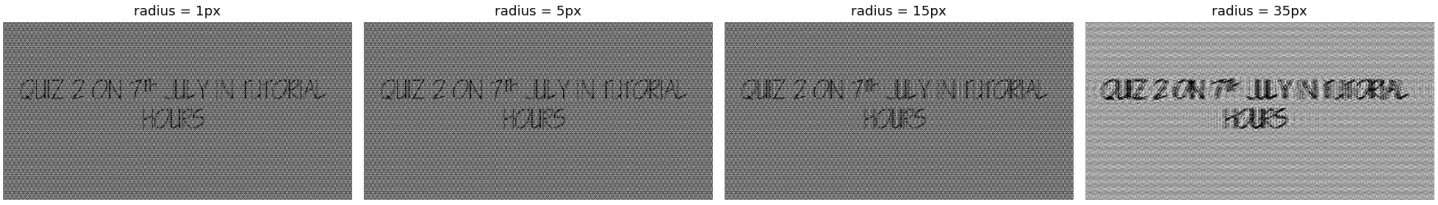


Figure 4: [Figure 4 goes here $-1 \times 4$ grid of recovered images at radius $= 1, 5, 15, 35$ pixels]

faint but noticeable grid texture on top of the recovered image. The text is still readable, but the corruption isn't fully cleaned up.

- **radius = 5px - the sweet spot**: At $r = 5$, the disk fully covers each spike and its nearby spread, but stays small enough that it doesn't touch the surrounding real image frequencies. This gives the cleanest output: sharp text, basically no visible grid, and no new artifacts. This is the radius we used for the main result.
- **radius = 15px - starting to cut into the signal**: At $r = 15$ the notch disks have grown big enough to start overlapping genuine image frequencies near each spike. Frequencies responsible for sharp text edges start getting zeroed along with the noise. In the spatial domain this shows up as a gradual softening/blurring of the text - still readable, but the edges have lost some sharpness. Even a "moderate" amount of over-filtering already costs something.
- **radius = 35px - Gibbs phenomenon**: At $r = 35$ the damage is much worse, and in a way that might be unexpected: instead of just blurring, the recovered image shows ringing - ghost edges, doubled outlines, wave-like oscillations spreading out from high-contrast regions. This is the Gibbs phenomenon. Zeroing out a big circular region in the frequency domain creates a sharp discontinuity - the spectrum jumps abruptly from its normal value to zero right at the circle's edge. Representing that kind of sharp step exactly would need sinusoids at arbitrarily high frequencies, but our image only has a finite number of frequency bins to work with. So when the inverse DFT tries to rebuild the image, it can't represent that discontinuity cleanly, and the result is oscillatory ringing spreading outward. The bigger the zeroed region, the sharper the discontinuity, and the worse the ringing. It's the same effect you see when a square wave is approximated with too few Fourier terms.

Main takeaway: removing more frequencies is not the same thing as removing more noise. Past the optimal radius, you start destroying real content and introducing new artifacts, which can actually make the image harder to read than a less aggressive filter would have.

| Radius | Noise Removed | Signal Preserved | Artefacts | Overall |
|---|---|---|---|---|
| $r = 1px$ | Incomplete - faint grid remains | Fully preserved | None | Suboptimal |
| $r = 5px$ | Complete | Fully preserved | None | Optimal ✓ |
| $r = 15px$ | Complete | Partially lost (edges) | Mild blurring | Acceptable |
| $r = 35px$ | Complete | Significantly lost | Gibbs ringing | Poor |

Table 2: Table 2: Summary of how filter radius affects recovery quality.

# 8 Discussion

## 8.1 Why does this actually work?

It's worth pausing to spell out clearly why frequency-domain filtering succeeds where a spatial-domain approach wouldn't. In the spatial domain, every pixel of the corrupted image is just the text pixel value plus the dot-grid pixel value added together. There's no way to look at one pixel and tell which part belongs to which - they're fully mixed. Any spatial-domain trick like smoothing or median filtering would just blend these values, which might reduce the noise a bit but would also blur the text, since it can't actually distinguish between them.

The frequency domain exposes something the spatial domain hides completely: the dot grid is periodic, and periodic signals are separable in frequency. The grid's energy sits on a tiny set of discrete points out of the whole 2D frequency plane, while the text's energy is spread out continuously. That lets us go in and remove just that tiny set of points, leaving almost the entire spectrum (and therefore almost the entire image) completely untouched.

### 8.2 Does removing more frequencies always help?

No, clearly not, as we saw in Section 7. There's an optimal amount of filtering, and both under-filtering (leftover noise) and over-filtering (destroyed signal + Gibbs ringing) give worse results than the well-tuned middle ground. The goal is to remove exactly the noise and nothing more - the smallest possible intervention in frequency space that still does the job.

### 8.3 Where else is this used?

This idea - move to frequency domain, find and remove the unwanted components, move back - isn't just a textbook exercise. It shows up in plenty of real systems:

- **Communication systems.** A received radio signal can pick up narrowband interference, e.g. a jamming tone at one fixed frequency, or harmonic interference from some piece of electrical machinery. Just like our dot grid concentrated all its energy at a few bins, a narrowband jammer concentrates its energy at one frequency. A notch filter there removes the jammer while leaving the broadband information signal around it intact - the exact same pipeline, just in 1D.
- **Remote sensing / satellite imaging.** Sensors that scan a scene line by line often produce periodic stripe artifacts at the scan rate. Because these stripes repeat at a fixed period, they show up as clear spikes in the 2D Fourier spectrum, exactly like our dot grid. Frequency-domain destriping is actually a standard step in processing pipelines for satellite imagery.
- **Medical imaging (MRI/CT).** MRI scanners switch magnetic field gradients at a fixed rate, and any imperfection or vibration in the coils introduces a periodic artifact at that rate - visible as stripes or ghosting in the image. Regular patient motion like breathing or heartbeat can do the same thing. Both show up as isolated spikes in k-space (the MRI version of the Fourier spectrum) and can be filtered out with notch filtering. The catch in medical imaging is that the filtering has to be conservative, since removing real diagnostic detail by mistake could mean missing a pathology.
- **Surveillance/signal intelligence.** The exact scenario in this question - a deliberately corrupted image hiding a message - is a realistic example of signal-intelligence work. People may corrupt images on purpose to stop interception or to hide information in plain sight, and frequency-domain analysis is a standard forensic tool for catching both.

In every one of these cases the principle is the same: find a domain where the unwanted structure becomes compact and identifiable, make a small targeted intervention, then come back. The Fourier domain is what makes periodic interference compact, which is exactly why the DFT shows up across so many different engineering fields.

## 9 Conclusion

This question showed how useful frequency-domain processing can be for image restoration. We started from an image that looked completely destroyed by a periodic dot-grid pattern, and by going through a four-step pipeline (transform → locate noise → notch filter → inverse transform) we recovered the hidden message.

Main points from this exercise:

- Periodic interference shows up as isolated spikes at the fundamental frequency and its harmonics - structurally very different from real image content, which is always spread continuously.
- The 2D DFT is the tool that gets us into this domain: `np.fft.fft2` to transform, `fftshift` to make it visually sensible, and `ifftshift` + `ifft2` to come back.

- A notch filter is the right choice because it removes exactly the noise frequencies (plus their conjugate mirrors) and leaves everything else alone.
- The filter radius needs careful tuning - too small and noise remains, too large and you start destroying real content and getting Gibbs ringing. $r = 5$ turned out to be the sweet spot for this image.
- You need to notch the harmonics too, not just the fundamental, since the dot pattern isn't a clean sinusoid and spreads energy across multiple harmonics.
- The recovered image clearly reveals the hidden message with the grid pattern essentially gone, which confirms the approach worked.
- This kind of frequency-domain recovery is directly relevant to real applications in communications, remote sensing, medical imaging and surveillance - anywhere periodic interference needs to be separated from genuine broadband information.